

Sequential Baseline: Linear Regression via Normal Equations Benchmarking Report

Ilyas Bouhafid

October 4, 2026

1 Experimental setup

- Machine: FT3
- Compilers: `gcc 10.1.0, icc 2021.3.0, icx 2021.3.0 (module load cesga/2020 intel/2021.3.0)`.
- Flags: `-O0, -O2 -march=native, -O3 -march=native, -Ofast -march=native`. Binaries built on the compute node.
- Each measurement is the average of 5 runs with seed 42 and noise $\sigma = 0.5$).
- Timed region: $X^\top X + X^\top y + \text{solve}$.

2 Verification

Config	$N \times p$	Max $ \beta - \beta_{true} $ ($\sigma = 0.5$)	Max $ \beta - \beta_{true} $ ($\sigma = 0$)
1	20000×50	0.008139	$< 10^{-6}$
2	50000×300	0.007840	$< 10^{-6}$
3	2000×2000	30.288211	10^{-6}

Table 1: Error against the ground truth. Identical for every compiler, flag and solver up to the last digits with `-Ofast`.

Configs 1 and 2 match β_{true} within the expected statistical error. Config 3 is a square system ($N = p$): the solution fits every noisy observation exactly, and the bad conditioning of $X^\top X$ makes the noise more important. With $\sigma = 0$ the error drops to 10^{-6} , confirming the implementation is correct.

3 Results

Compiler	Flags	Config 1	Config 2	Config 3
gcc	-O0	0.267	29.686	42.619
gcc	-O2	0.209	19.351	16.656
gcc	-O3	0.210	19.378	15.753
gcc	-Ofast	0.210	19.374	15.770
icc	-O0	0.263	31.735	40.441
icc	-O2	0.201	19.071	14.568
icc	-O3	0.201	19.100	14.789
icc	-Ofast	0.201	19.065	14.791
icx	-O0	0.321	30.205	41.445
icx	-O2	0.226	19.744	17.919
icx	-O3	0.227	19.756	17.929
icx	-Ofast	0.242	20.383	17.806

Table 2: Total time, Gaussian elimination.

Compiler	Flags	Config 1	Config 2	Config 3
gcc	-O0	0.268	29.604	47.550
gcc	-O2	0.210	19.338	17.121
gcc	-O3	0.210	19.351	16.372
gcc	-Ofast	0.216	19.309	16.378
icc	-O0	0.263	29.605	45.463
icc	-O2	0.201	19.147	15.235
icc	-O3	0.202	19.910	15.312
icc	-Ofast	0.201	19.416	15.332
icx	-O0	0.302	29.889	46.638
icx	-O2	0.236	20.135	18.517
icx	-O3	0.236	19.761	18.356
icx	-Ofast	0.246	20.567	18.069

Table 3: Total time, Gauss-Jordan.

Compiler	Flags	$X^\top X$	Solve (GE)	Solve (GJ)	GJ/GE
gcc	-O0	32.620	9.984	14.977	1.50
gcc	-O2	14.615	2.034	2.494	1.23
gcc	-O3	14.616	1.131	1.742	1.54
gcc	-Ofast	14.638	1.128	1.743	1.55
icc	-O0	31.195	9.231	13.819	1.50
icc	-O2	13.644	0.919	1.432	1.56
icc	-O3	13.871	0.912	1.441	1.58
icc	-Ofast	13.872	0.913	1.446	1.58
icx	-O0	31.809	9.621	14.467	1.50
icx	-O2	16.978	0.932	1.495	1.60
icx	-O3	16.980	0.939	1.499	1.60
icx	-Ofast	16.845	0.951	1.507	1.58

Table 4: Config 3 breakdown : $X^\top X$ time and solve time for both solvers.

4 Observations

- **Flags:** `-O2` gives $1.3\text{--}2.8\times$ over `-O0`. `-O3/-Ofast` add almost nothing: the column-wise access to X is memory-bound. Exception: gcc's solver only vectorizes at `-O3` ($2.03\text{ s} \rightarrow 1.13\text{ s}$).
- **Compilers:** equal at `-O0`. `icc` fastest, `icx` slowest on $X^\top X$ (config 3: 14.6 vs 17.9 s). Config 2: all $\approx 19\text{--}20\text{ s}$.
- **Memory:** config 3 runs $X^\top X \approx 2\times$ faster per operation than configs 1–2, since its short columns (2000 rows) stay in cache.
- **Solvers:** negligible in configs 1–2; up to 23% of total in config 3. Gauss-Jordan $\approx 1.5\times$ slower ($p^3/2$ vs $p^3/3$).